

Introduction

For this semesters lab term project we have created 3 different types of RISC-V compliant computer processors. These computer processors are a single cycle processor, a software scheduled pipelined processor and lastly a hardware scheduled pipelined processor. A little bit more about these processors: Although they execute the same RISC-V instruction set, each design takes a different approach to instruction throughput, control logic, and timing. The most straightforward of the three in our opinion is the single cycle processor. This processor operates by fetching and fully executing each instruction once per clock cycle. The software scheduled processors is slightly different it splits the execution of each instruction into stages. These stages consist of Instruction Fetch, Instruction Decode, Execute, Memory and Write Back stages. This can introduce data and control hazards withing the processor that are handled in software by inserting NOP's. This is exactly where the third processor comes in. The Hardware scheduled processor handles these data and control hazards by itself. It does this by using data forwarding and implementing stalls that let dependent instructions hold their position in the pipeline until the hazard is resolved, ensuring that the processor always uses the correct values. These three processors illustrate some of the many different strategies to execute the same instruction set with different levels of efficiency and hardware complexity.

Bench Marking

Single Cycle Processor

Programs	# of instructions	# of cycles	CPI	Max Cycles Time	Total Execution time
Base Test	58	58	1	42.732 ns	58 x 42.732 ns = 2,478.456 ns
Control Flow Test	31	31	1	42.732 ns	31 x 42.732 ns = 1,324.692 ns
Grendel	2129	2129	1	42.732 ns	2129 x 42.732 ns = 90976.428 ns
MergeSort	2605	2605	1	42.732 ns	2605 x 42.732 ns = 111,316.86 ns

Software Scheduled Cycle Processor

Programs	# of instructions	# of cycles	CPI	Max Cycles Time	Total Execution time
Base Test	62	65	1.05	34.649 ns	65 x 34.649 ns = 2,252.185 ns
Control Flow Test	37	40	1.08	34.649 ns	40 x 34.649 ns = 1,385.96 ns
Grendel	5427	5430	1.00055	34.649 ns	5430 x 34.649 ns = 188,144.07 ns
MergeSort	4066	4069	1.00074	34.649 ns	4069 x 34.649 ns = 140,986.781 ns

Hardware Scheduled Cycle Processor

Programs	# of instructions	# of cycles	CPI	Max Cycles Time	Total Execution time
Base Test	58	61	1.05	35.064 ns	61 x 35.064 ns = 2,138 ns
Control Flow Test	31	48	1.55	35.064 ns	48 x 35.064 ns = 1,683.072 ns
Grendel	2129	2863	1.34	35.064 ns	2863 x 35.064 ns = 100,388.232 ns
MergeSort	2605	2939	1.13	35.064 ns	2963 x 35.064 ns = 103,053.096 ns

Performance Analysis

In order to understand the trade offs and design decisions of the different processor designs this section compares the behavior of the single cycle, software scheduled pipelined, and hardware scheduled pipelined processors across four different test programs. These 4 programs include: Base Test, that test all R type and I Type instructions that do not effect the PC aside from moving to the next instruction. Control Flow Test a program that runs every branch and jump instruction. Grendel a CPU stress test program that includes lots of recursion, branches, function calls, and bit operations. Lastly Merge Sort that relies heavily on recursion, and memory addressing for arrays in memory. Together, these programs provide a comprehensive benchmark suite that exposes each processor's strengths and weaknesses, allowing for a clear comparison of their performance under diverse workloads.

To generate the performance table from running the four testing programs. The python code gave us both the number of instructions as well as number of cycles ran during each program for each processor. We then can use these values to calculate The CPI for a the program that was just ran by dividing the number of cycles the processor took to complete the program by the number of instructions in the program being run. Once we have the CPI from these calculations we then synthesize each processor. Synthesizing a processor takes the VHDL implementation of the processor and turns it into an actual hardware implementation that could run on a fabricated chip. The synthesization also gives you the max clock cycle time. With all this information to get the total execution time of the program one must multiply the max clock cycle time by the number of cycles for the program. This final execution time provides the foundation for comparing the processors, revealing how design choices impact real-world performance.

The first processor analyzed is the single-cycle design. Across the four benchmark applications, it preformed mediocre compared to the Software scheduled pipeline processor and far worse that the hardware scheduled pipeline. The single cycle processor has many different strengths and weaknesses that make it good for different applications. One very strong trait a single cycle processor possesses it the ability to execute every instruction in one cycle. This eliminates the need for any stalling or forwarding for branch or jump instructions which is both helpful for execution times of programs that have a lot of branching such as the Control Flow Test, Grendel and Merge Sort programs and hardware simplicity. As you can see compared to the software scheduled program the total execution times for those programs compared to the single cycle are much higher due to NOPS inserted to avoid data and control hazards. With that being said the main draw back that makes the single cycle processor limited in its ability compared to the pipelined processors is the max clock cycle time. This maximum clock cycle time is determined by the longest instruction in the instruction set because every instruction must complete with in a single

cycle. As a result, the clock period for the single-cycle processor is much longer than that of the pipelined processors, which can break instruction execution into smaller stages (43 ns compared to 35 ns). Even though it avoids stalls, the longer cycle time limits overall throughput, making the single-cycle processor slower on programs with many instructions. Therefore, the single cycle processor excels in simplicity and branch handling, but its inherently long cycle time becomes the primary hindrance in performance.

The second processor analyzed is the software scheduled pipeline processor. Across the four benchmark applications, it performed the worst overall out of the three different processors. The only instance the software processor performed better than either of the other two processors was the Base Test and Control Flow Test beating out the single cycle processor very minimally. This performance can be largely explained by the trade offs in the design characteristics of the software-scheduled pipeline. While a software scheduled pipelined processor can have multiple instructions being executed in different stages of the processor at the same time, thus essentially cutting the critical cycle time with registers, there are some downsides. The downside of the software scheduled pipelined processors is that there is no forwarding or stalls inserted through hardware meaning that in many of the test programs, NOPS had to be inserted to alleviate all of the data and control hazards. The insertion of necessary NOPS into the programs increases the number of instructions so much so that the reduction in maximum cycle time was not worth pipelining the processor for both merge sort and Grendel.

The third and final processor analyzed is the hardware scheduled pipelined processor. In three of the four benchmark programs the hardware scheduled processor performed the best out of all the processors. The only program that the hardware scheduled processor was not the best in was the control flow test due to the processor still needing to stall one time in order to execute branch instructions. The hardware scheduled processor uses extra hardware units to forward and stall in order to resolve hazards dynamically. This ability to detect and forward away to resolve data hazards allows it to keep the pipeline full and minimize unnecessary stalls. As one could see from the tables above the hardware scheduled pipelined processor has a significant less number of cycles in comparison to the software scheduled pipelined processor this is due to the forwarding ability. Another strength that this processor has is the fact that it is still pipelined with registers. Referencing the tables again the hardware scheduled pipelined processor has a significantly shorter maximum cycle time compared to the single cycle processor and has a comparable max cycle time to the software scheduled pipelined register. Together, these features explain why the hardware scheduled pipeline consistently delivers the best overall performance. The hardware scheduled pipelined processor combines the short cycle time of a pipelined design with intelligent hazard handling that keeps the pipeline busy and reduces wasted cycles.

Overall, the results of these four benchmark programs highlight how each processor's design directly shapes its real-world performance. The single-cycle processor benefits from simplicity and avoiding stalls entirely, but its long clock period ultimately limits throughput on larger workloads. The software-scheduled pipeline improves cycle time through pipelining but loses most of those gains to the large number of compiler-inserted NOPs required to handle data and control hazards. This downfall in our programs leading to the weakest overall results. In contrast, the hardware-scheduled pipeline consistently achieves the best performance by combining a short clock period with dynamic hazard detection and forwarding, which keeps the pipeline filled and minimizes wasted cycles. As a result, the hardware-scheduled pipeline we believe is the most balanced and efficient design among the three processors analyzed.

Software Optimization

A key software-level optimization that improves performance on a software-scheduled pipeline processor is instruction scheduling to avoid data dependencies instead of inserting 2-3 NOPS between instructions that contain data dependencies. If this was done for the programs in the software scheduled pipelined processor it would greatly reduce the number of instructions in the program thus in turn also reducing the number of cycles and total execution. An estimate for performance improvement for Base Test assuming 10% of instructions are load-use hazards, each hazard normally causes a 2-cycle stall. Software scheduling removes 70–80% of those stalls depending on available independent instructions. You may have a Performance gain of 20-30% which can be seen in the performance change from the software scheduled pipelined processor to hardware scheduled as hardware scheduled processor removes those data hazards by itself.

Hardware Optimization

1. Single-Cycle

An improvement to the single-cycle pipeline that we applied to our scheduled-pipelines would be using faster adders like lookahead adders or an even more advanced design. Our largest unit in our critical path was by far our Adders, whether that was in the ALU or in the PC Fetch Unit. By improving the speed of these adders, we would be able to improve our critical path, provided that the new adder design was faster than ~14ns, the current speed of our adders.

2. SW-Scheduled Pipeline

A technical improvement that we could make to our SW pipeline would be to change the clock edge triggers of our pipeline registers from negative to positive. Our current method, while it has benefits, has heavy penalties. The current state of the registers causes issues with synthesis, causing us to lose half of the cycle and add a 10ns penalty off the bat because of negative edge triggering. By changing this to positive edge triggering, we remove the immediate penalty and set up our processor to synthesize with a performance increase.

3. HW-Scheduled Pipeline

An improvement to our HW-scheduled pipeline would be to implement another register between the ID and EX stages, to break up our jumping and branching instructions. Our current critical path is heavily limited by the fact that our PC-updating instructions are computed and completed inside of a singular ID stage. This operation, while it has its benefits, caps our maximum cycle time and slows down our processor. By computing the PC/REG + IMM operation in the ID stage, and then updating the PC in our EX stage, we would be able to split the largest functional unit into a separate stage, hopefully improving our critical path. The improvement would be unknown, as it would be fully changing the critical path. This would likely change the processor's critical path to be the EX or MEM stage, depending on if DMem or the ALU as a longer cycle time.

It Depends

A program that will run better on the single cycle processor over our hardware scheduled pipelined processor would be a program that has a lot of repeated branches with limited I-Type and R-type instructions. This type of program will run faster on a single cycle processor over the hardware processor because the single cycle executes every branch and jump instruction in 1 cycle no matter what. This is different from a hardware scheduled pipeline. In order to know if an instruction is taking a branch or not in a hardware scheduled pipelined processor that instruction must propagate from the IF stage into the ID stage where it is then handled and the PC can be updated appropriately. This means that the hardware scheduled pipeline must stall for one cycle on every branch instruction increasing the number of cycles compared to the single cycle processor. Along with this the program will need to have a limited amount of I-Type and R-type instructions because that is where the hardware scheduled pipeline processor has the advantage as it has a

shorter max cycle time and every data hazard with I-type and R-type instructions can be directly handled with data forwarding.

A program that runs faster on the software scheduled pipeline is the control flow tests. This is due to the same reason as before if the program for software is written with as little NOPS as possible and lots of branches the hardware scheduled pipeline will have to stall where the software would not. Along with this the cycle time for the software is faster so every instruction would be executed faster as well.

Challenges for our Group:

1. Barrel Shifter

When constructing our barrel shifter, we had several problems with understanding the instructions and with fully grasping the technical requirements of the unit. During our initial construction of the unit, we failed to understand that it had to be written using structural VHDL. This required us to rewrite the unit, and create the underlying logic for the unit. The barrel shifter was an easy trap to fall into, using the libraries to shortcut writing the structural code for the unit also created a failure to understand the technical specifications necessary. Our shifter, written with the more advanced libraries did not actually fulfill the technical requirements, as it would not wrap around when given values over 32. This situation could have been avoided, saving us time and effort by more thoroughly reading the project documentation and by understanding the technical requirements of the barrel shifter.

2. Writing the report for Project 2

In a similar situation to the barrel shifter, we did not read the project 2 documentation as thoroughly as we should have before we started working. While this ultimately did not have as detrimental an effect on our processor's creation and functionality, it became a much larger issue when we began writing the report. Once we finished both the SW and HW scheduled pipelines and we began working on the reports, it very rapidly became apparent that we should have been working on the report throughout our development period. Beginning work on the report after our processor was finished created several pros and cons. In favor of the approach, we were able to complete the entire report in a single session, as we had all the data available to us. Against the approach, it slowed us down significantly when we had to go back and develop test-benches and lists that would have been beneficial, if not necessary, in our development process. This challenge could have been lessened by continuing the report in parallel with our development process, and although it wasn't necessary for us as a group, it would have been helpful.

3. Rewriting the Grendel and Merge sort programs for software scheduling

A purely technical challenge that was more frustrating and annoying than difficult was rewriting the test programs to be software pipelined. While attempting to the best of our abilities to follow the requirements and reduce the number of required NOP instructions, it became readily apparent that rewriting the Mergesort program was the best option. While it was frustrating, it was not nearly as challenging as managing the provided grendel program. After reviewing the grendel program and failing to modify it multiple times, we came to the conclusion that it was necessary to just use NOPs to avoid hazards. Grendel was challenging to rewrite due to its structure. The labels and program flow were difficult to understand, the code was written without much regard for proper ordering, and the usage of registers was strange to us. Ultimately, the challenge was unavoidable and had to be persevered through.

Demo

During our demo and speaking with Professor Duwe, we began to understand that some of our supposed strengths were actually weaknesses. Our design completed branch and jump instructions within the Instruction Decode stage, which gives our processor only a half cycle to path through all the necessary components, including an adder, which drove our cycle time up dramatically. A method of resolving these issues while maintaining our current pipeline design, would be to separate the computation and resolution of Branch/Jump instructions into the execute stage, dividing the critical path over a clock edge and hopefully improving the frequency. This would break up the critical path and improve our frequency at the penalty of needing to flush the IF/ID register. This should be a worthwhile tradeoff, as the performance increase from modifying the critical path should offset the time penalties from needed to flush instructions on a taken branch/jump.